Re-Id

Friday, April 24, 2026 11:16 AM

Cursor:

"I built a real-time person Re-ID tracking pipeline with YOLOX detection, ByteTrack tracking, and OSNet Re-ID. The architecture splits detections into person (Re-ID path with embedding matching) vs non-person (pure tracking). All components are integrated into a FastAPI backend with WebSocket streaming to an HTML UI. The pipeline works but shows 0 detections because I'm using random YOLOX weights — I need to load my trained model into `weights/yolox.pth`. Key files: `core/detector.py` (YOLOX), `core/tracker.py` (ByteTrack), `core/reid.py` (OSNet), `core/memory.py` (embedding gallery), `core/pipeline.py` (orchestrator), `api/server.py` (backend), `ui/index.html` (frontend), `configs/pipeline_config.yaml` (config)."

Architecture Overview

Built a **real-time person Re-ID + multi-object tracking pipeline** with this flow:

```
Input (video/URL/frames)
      ↓
   YOLOX Detection (8 classes: forklift, truck, no_truck, person, box_in_hand, palet_load, gateopen, gateclose)
   Outputs: bbox, class_id, confidence
      ↓
  ┌──────────────┴──────────────┐
  |                             |
person class?          non-person classes?
  |                             |
  ↓                             ↓
ByteTrack                ByteTrack
(Motion + IoU)           (Motion + IoU only)
  ↓                             |
Crop person                     |
  ↓                             |
OSNet Re-ID                     |
Extract 512-d embeddings        |
  ↓                             |
ReID Memory Store               |
Cosine similarity matching      |
Assign person_1, person_2...    |
  ↓                             |
  └──────────────┬──────────────┘
          ↓
      FastAPI Backend
          ↓
      WebSocket Stream
          ↓
        HTML UI
   (shows annotated video with bboxes,
    track IDs, person IDs, confidence)
```

Project Structure

```
reid_pipeline/
├── core/
```

```
│   ├── detector.py        # YOLOX wrapper - detects 8 classes, splits person vs non-person
│   ├── tracker.py         # ByteTrack wrapper - PersonTracker (with crops) + ObjectTracker
│   ├── reid.py            # OSNet feature extractor - 512-d L2-normalized embeddings
│   ├── memory.py          # ReID gallery - stores embeddings, cosine matching, assigns person_N IDs
│   ├── pipeline.py        # Orchestrator - connects all components per frame
│   └── bytetrack/         # ByteTrack source files (byte_tracker.py, kalman_filter.py, basetrack.py,
matching.py)
├── input/
│   └── source.py          # (empty - not implemented yet)
├── api/
│   └── server.py          # FastAPI backend - /upload endpoint, /ws WebSocket for real-time
├── ui/
│   └── index.html         # Frontend - video viewer, controls, stats, active tracks sidebar
├── weights/
│   └── (put yolox.pth and osnet.pth here)
├── configs/
│   └── pipeline_config.yaml # All config params
├── YOLOX/                 # YOLOX library (cloned from GitHub)
├── ByteTrack/             # ByteTrack library (cloned, files copied to core/bytetrack/)
└── main.py                # Entry point
```

Key Components Implemented

1. Detector (`core/detector.py`)

- Loads YOLOX model with 8 custom classes
- `detect(frame)` → returns `(persons, non_persons)` lists
- Each detection: `{bbox, conf, class_id, class_name}`
- Person class_id = 3

2. Tracker (`core/tracker.py`)

- **PersonTracker**: ByteTrack for persons, returns crops for Re-ID
- **ObjectTracker**: ByteTrack for non-persons (forklift, truck, etc.)
- Both use Motion + IoU matching
- Returns tracks with stable `track_id`

3. Re-ID (`core/reid.py`)

- OSNet (osnet_x1_0) pretrained on ImageNet
- `extract(crop)` → 512-d L2-normalized embedding
- Preprocesses: resize to (256, 128), normalize with ImageNet stats

4. Memory (`core/memory.py`)

- Stores up to 10 embeddings per person_id
- `match(embedding)` → compares with gallery using cosine similarity
- Threshold: 0.65 (configurable)
- Assigns `person_1, person_2, ...` IDs

5. Pipeline (`core/pipeline.py`)

- `process_frame(frame)`:
  1. Detect → persons + objects
  2. Track persons → get crops
  3. Extract embeddings from crops
  4. Match embeddings → assign person_id
  5. Track objects (no Re-ID)
  6. Return `{persons: [...], objects: [...]}`
- `draw(frame, result)` → annotate with green boxes (persons) + blue boxes (objects)

6. API (`api/server.py`)

- **GET /**: Serves HTML UI
- **POST /upload**: Saves video file
- **WebSocket /ws**:
  - Accepts `{type: "file", path: "..."}` or `{type: "url", url: "..."}`
  - Processes video frame-by-frame
  - Sends `{image: base64, persons: [...], objects: [...]}`

7. UI (`ui/index.html`)

- Dark theme with cyan accents
- Video canvas (left), sidebar (right)
- Real-time stats: Persons, Objects, FPS, Frames
- Active tracks list with color-coded boxes
- Controls: URL input, file upload, start/stop

Dependencies Installed

```
torch, torchvision       # Deep learning
opencv-python            # Video I/O
numpy, scipy             # Math
torchreid                # OSNet Re-ID
lap, filterpy, motmetrics # ByteTrack dependencies
scikit-image, h5py       # Image processing
fastapi, uvicorn         # Backend
websockets               # Real-time streaming
PyYAML                   # Config
```